

Image Blurring by Spatial Filtering and its Fourier-Domain Equivalent

CSc 8830 Computer Vision | Module 3 Assignment

Student: **(your name)** GitHub repository:

[https://github.com/\(your-username\)/cv-module3-blur](https://github.com/(your-username)/cv-module3-blur)

1. Summary

I blurred images with a normalised box filter and a Gaussian filter in two ways: (a) direct convolution in the spatial domain, coded from the definition without any library convolution, and (b) multiplication of Fourier transforms followed by an inverse transform. Across every kernel tested, the two results agree to within floating-point rounding: the largest difference for a 15×15 Gaussian on a 512×512 image is **4.83e-13** gray levels on a 0 to 255 scale, about twelve orders of magnitude below one quantisation step (one gray level). Section 3 proves the convolution theorem and Sections 4 to 6 validate it experimentally. The one condition that must hold is zero-padding before the FFT; without it the two methods disagree near the borders by up to **118** gray levels.

2. Implementation

Everything is in `blur_filters.py` (with a ReadMe header) and is exposed through the Flask web app `app.py`. Both images below use the same kernels and the same zero-padding assumption (pixels outside the image are 0).

Kernels. The box filter has all K^2 weights equal to $1/K^2$. The Gaussian is $h[m,n] \propto \exp(-(m^2+n^2)/2\sigma^2)$, sampled on a $K \times K$ grid and normalised. Both kernels sum to 1, so mean brightness is preserved (an un-normalised 5×5 box would make the image 25× brighter and saturate it).

Spatial filtering. Convolution is implemented straight from its definition:

$$g[i,j] = \sum_a \sum_b h[a,b] f[i-a, j-b]$$

Each kernel weight $h[a,b]$ contributes a copy of the image shifted by (a,b) ; adding the K^2 shifted copies is exactly the flip-and-slide sum (the flip is built into the index $i-a, j-b$). The full result has size $(H+K-1) \times (W+K-1)$ and is cropped back to $H \times W$. As a check, the Gaussian is separable, and two 1-D passes give the same answer as the 2-D kernel to 5.4e-13.

Fourier filtering. Zero-pad the image and the kernel to the same size (at least $(H+K-1) \times (W+K-1)$; the code rounds up to an FFT-friendly size), take the 2-D FFT of each, multiply them element by element, and take the inverse FFT. In code:

```
F = fft2(f, s=shape); Hk = fft2(h, s=shape); g = real(ifft2(F * Hk))
```

Independent reference. SciPy's `convolve2d` is used only as a third opinion; it agrees with both methods (Table 1).

3. Theory: convolution in space equals multiplication in frequency

3.1 Continuous form. Let $g(x) = (f * h)(x) = \int f(\tau) h(x-\tau) d\tau$ and let the Fourier transform be $F(u) = \int f(x) e^{-j2\pi ux} dx$. Then

$$G(u) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(\tau) h(x-\tau) e^{-j2\pi ux} d\tau dx$$

Exchange the order of integration and write $e^{-j2\pi ux} = e^{-j2\pi u\tau} \cdot e^{-j2\pi u(x-\tau)}$.

$$G(u) = \int f(\tau) e^{-j2\pi u\tau} d\tau \cdot \int h(x-\tau) e^{-j2\pi u(x-\tau)} dx$$

In the second integral substitute $\mu = x-\tau$. Because τ is finite the limits stay $(-\infty, \infty)$, so it equals $H(u)$, the transform of h . The first integral is $F(u)$. Therefore

$$G(u) = F(u) H(u) \quad \Longleftrightarrow \quad g = f * h$$

The same derivation in two dimensions gives $G(u,v) = F(u,v) H(u,v)$. Running the argument in the other direction shows that a product in space is a convolution in frequency.

3.2 Discrete form (what the code uses). For $P \times Q$ arrays, the DFT is $F[u,v] = \sum_m \sum_n f[m,n] e^{-j2\pi(um/P + vn/Q)}$, and the product of DFTs corresponds to *circular* convolution:

$$g_c[i,j] = \sum_{m=0}^{P-1} \sum_{n=0}^{Q-1} f[m,n] h[(i-m) \bmod P, (j-n) \bmod Q]$$

Take its DFT, swap the sums, and put $i' = (i-m) \bmod P$ and $j' = (j-n) \bmod Q$. As i runs over $0 \dots P-1$, so does i' (a bijection), and the complex exponential is periodic in P , so $e^{-j2\pi ui/P} = e^{-j2\pi ui'/P}$. $e^{-j2\pi um/P}$. The sum splits:

$$G_c[u,v] = \left(\sum_{m,n} f[m,n] e^{-j2\pi(\frac{um}{P} + \frac{vn}{Q})} \right) \left(\sum_{i',j'} h[i',j'] e^{-j2\pi(\frac{ui'}{P} + \frac{vj'}{Q})} \right) = F[u,v] H[u,v]$$

3.3 Why zero-padding makes it equal to spatial filtering. An image of height H and a kernel of height K , both zero outside their support, are padded to $P \geq H+K-1$ rows. In the circular sum, whenever $i-m < 0$ the index wraps to $(i-m)+P \geq P-H+1 \geq K$, and h is zero at indices $\geq K$. So every wrapped term is zero, the circular sum equals the ordinary sum, and $\text{IFFT}(F \cdot H) = f * h$ exactly (same argument for columns). Padding to a smaller size lets the tail of the blur wrap around to the opposite edge, which is what Section 5.3 shows.

3.4 What the two kernels look like in frequency. The transform of a box of width K is a (discrete) sinc $|\sin(\pi u K) / (K \sin \pi u)|$, with side lobes. The transform of a Gaussian of spread σ pixels is another Gaussian, $H(u) = \exp(-2\pi^2 \sigma^2 u^2)$, of spread $1/(2\pi\sigma)$ in frequency, so a wider blur in space is a narrower low-pass in frequency. Both are low-pass filters; multiplying by them attenuates high frequencies, which are the edges and fine detail, and this is what blurring is.

3.5 Worked example (by hand, checked by computer)

Signal $f = [2, 4, 1, 3]$ and kernel $h = [1, 2, 1]$ (un-normalised so all numbers are integers). **Spatial:** $g[i] = \sum_a h[a] f[i-a]$.

```
g[0] = 1·2 = 2
g[1] = 1·4 + 2·2 = 8
g[2] = 1·1 + 2·4 + 1·2 = 11
g[3] = 1·3 + 2·1 + 1·4 = 9
g[4] = 2·3 + 1·1 = 7
g[5] = 1·3 = 3
g = [2, 8, 11, 9, 7, 3] (length 4+3-1 = 6).
```

Fourier: pad both to $N = 6$, take the 6-point DFT of each, multiply, and invert. The table lists each frequency k .

k	F[k]	H[k]	G[k] = F[k]·H[k]
0	10.000	4.000	40.000
1	0.500 - 4.330j	1.500 - 2.598j	-10.500 - 7.794j
2	2.500 - 2.598j	-0.500 - 0.866j	-3.500 - 0.866j
3	-4.000	0.000	0.000
4	2.500 + 2.598j	-0.500 + 0.866j	-3.500 + 0.866j
5	0.500 + 4.330j	1.500 + 2.598j	-10.500 + 7.794j

Inverse DFT of G : [2.000, 8.000, 11.000, 9.000, 7.000, 3.000] — identical to the spatial result [2, 8, 11, 9, 7, 3].

Without padding ($N = 4$, the length of f): the circular result is [9, 11, 11, 9]. The last two spatial outputs (7 and 3) wrapped around and were added to positions 0 and 1 ($2+7 = 9$, $8+3 = 11$). This is the same effect that appears at image borders in Section 5.3.

4. Experiment: spatial versus Fourier result

Test image: 512×512 grayscale “cameraman” (from scikit-image). The figure shows the original, the spatially blurred image, the Fourier-blurred image, and the absolute difference on a scale of 0 to 10^{-9} gray levels. The two blurred images are visually identical and the difference is rounding noise.



Figure 1. Gaussian (top) and box (bottom) blur of the same image, computed both ways.

Image	Kernel	max spatial-FFT	RMSE	max spatial-SciPy
cameraman (gray)	box 3x3	1.71e-13	3.81e-14	0.00e+00
cameraman (gray)	box 9x9	5.12e-13	8.94e-14	4.55e-13
cameraman (gray)	box 21x21	2.16e-12	7.11e-13	2.05e-12
cameraman (gray)	gaussian 5x5, s=1.0	1.99e-13	4.54e-14	1.14e-13
cameraman (gray)	gaussian 15x15, s=2.5	4.83e-13	9.13e-14	4.26e-13
cameraman (gray)	gaussian 31x31, s=5.0	9.95e-13	1.87e-13	1.02e-12
astronaut (RGB)	box 3x3	1.99e-13	3.50e-14	0.00e+00
astronaut (RGB)	box 9x9	4.26e-13	5.64e-14	3.98e-13
astronaut (RGB)	box 21x21	2.33e-12	4.65e-13	2.33e-12
astronaut (RGB)	gaussian 5x5, s=1.0	2.27e-13	4.08e-14	1.42e-13
astronaut (RGB)	gaussian 15x15, s=2.5	4.26e-13	7.14e-14	3.98e-13
astronaut (RGB)	gaussian 31x31, s=5.0	9.09e-13	1.30e-13	8.81e-13

Table 1. Errors in gray levels (0-255 scale) for several kernels and sizes, on a grayscale and an RGB image (each channel filtered separately). Every entry is at most 2.3e-12, i.e. floating-point rounding (the errors grow slightly with kernel size because more terms are summed).

5. Evidence from the frequency domain

5.1 The kernels are low-pass filters. The numerically computed spectra of the 15×15 kernels match the analytic transforms from Section 3.4 (largest Gaussian deviation 4.1e-03, from truncating the Gaussian to 15 taps). The box has side lobes; the Gaussian does not, which is why box blur looks blocky (it leaks some high frequencies back in) and Gaussian blur looks natural.

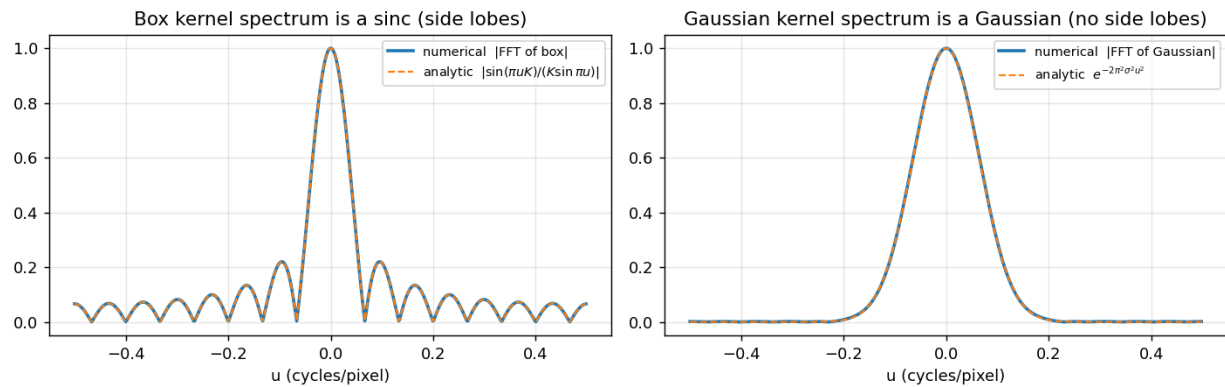


Figure 2. Kernel spectra (numerical solid, analytic dashed).

5.2 Multiplying the spectra is the blur. Left: spectrum of the original image, concentrated at low frequencies with strong axis-aligned streaks from the image edges. Middle: the Gaussian's spectrum, a small bright disc around zero frequency. Right: their product, which keeps the low frequencies and suppresses the rest. The inverse transform of the right panel is the blurred image.



Figure 3. $F(u,v)$, $H(u,v)$ and $G(u,v) = F(u,v)H(u,v)$, log-magnitude, zero frequency at the centre.

5.3 What goes wrong without zero-padding. If the FFT is done at the image size (no padding), the result is a circular convolution. Interior pixels still match to $5.4e-13$, but pixels within half a kernel width of an edge are blended with the opposite side of the image: the largest error is 117.9 gray levels and the mean error in the 7-pixel border is 21.10. This confirms the condition derived in Section 3.3.

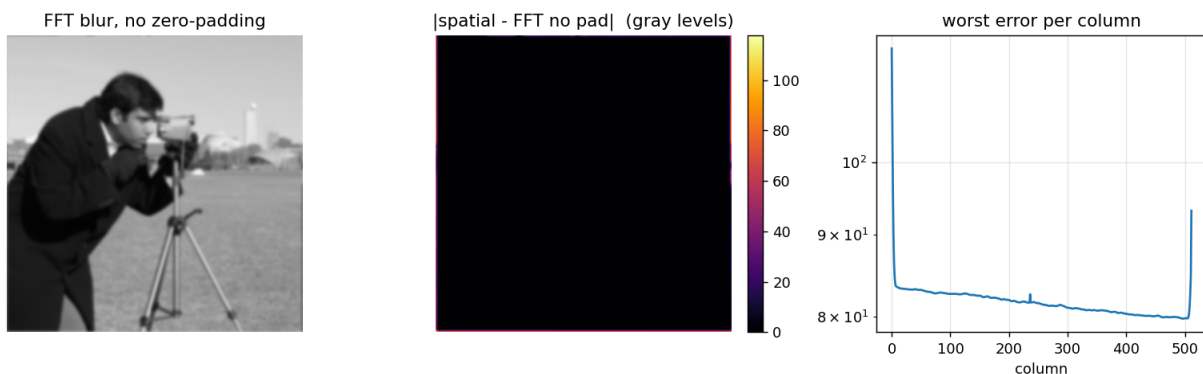


Figure 4. Un-padded FFT blur: error is confined to the borders.

6. Cost: when is the Fourier route worth it?

The direct spatial method needs K^2 multiplications per pixel ($2K$ if separable); the FFT route costs about the same whatever K is, because the transform size only depends on the image. Timings for this NumPy/SciPy implementation on a 512×512 image are below; absolute numbers depend on the machine, and a compiled or vectorised convolution would move the crossover point, but the trend is the general one.

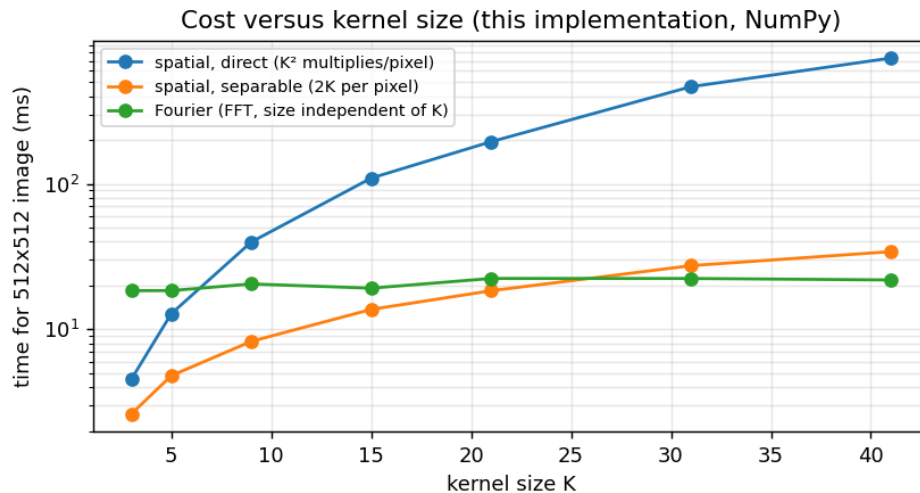


Figure 5. Time versus kernel size.

For $K = 3$ the direct method takes 5 ms and the FFT 18 ms; for $K = 41$ the direct method takes 732 ms versus 22 ms. The separable Gaussian (34 ms at $K = 41$) is far cheaper than the 2-D direct version, as the lecture notes on separability predict, but it still grows linearly with K while the FFT stays flat.

7. Web application demonstration

The web app (python app.py, then <http://127.0.0.1:5000>) lets the user pick the kernel, size, sigma, and image, and shows the spatial result, the Fourier result, their difference, the spectra, and the error metrics. A checkbox disables zero-padding to reproduce the wrap-around error. A screen recording of the working system is submitted with this report.

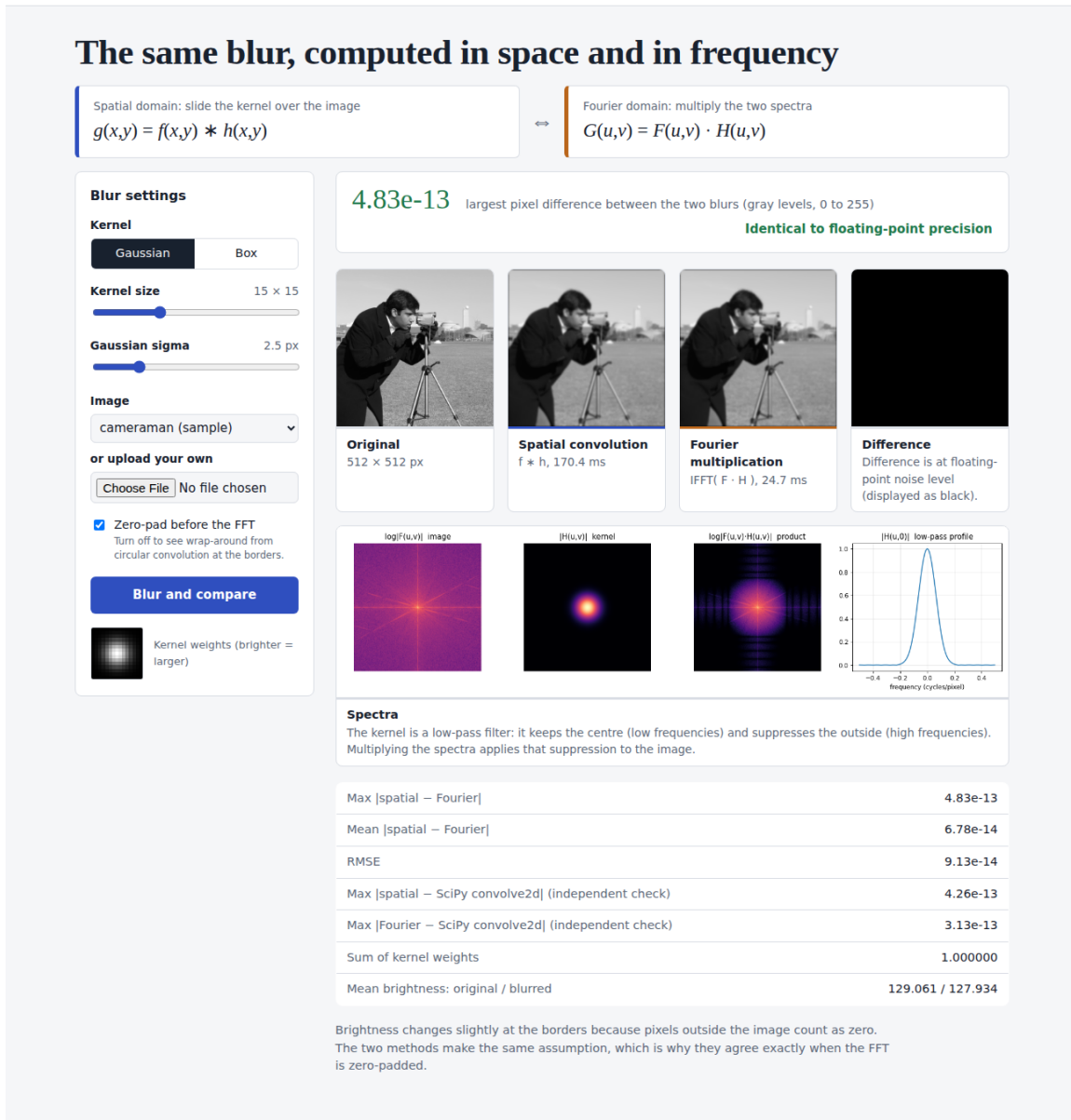


Figure 6. Screenshot of the web application.



Figure 7. Screenshot of the web application.

8. Conclusions

(1) Blurring with a normalised box or Gaussian kernel is a linear shift-invariant operation, i.e. a convolution. (2) The convolution theorem $G = F \cdot H$, proved above in continuous and discrete form, means the same blur can be computed by multiplying spectra. (3) Experimentally, the two routes agree to within $2.3e-12$ gray levels for every kernel tested, which is floating-point rounding. (4) Equality holds for the ordinary (linear) convolution only if both arrays are zero-padded to at least $(H+K-1) \times (W+K-1)$; otherwise the DFT's circular convolution wraps the blur across the image edges. (5) The Fourier view also explains *why* the blur looks the way it does: the kernels are low-pass filters, and the Gaussian's smooth spectrum avoids the side-lobe artifacts of the box.

References

Nayar, S. K., *Image Processing I* (FPCV-1-4) and *Image Processing II* (FPCV-1-5), First Principles of Computer Vision, Columbia University, 2022. The kernels, the convolution definition and the convolution-theorem derivation follow these lectures. The zero-padding condition in 3.3 is an addition needed for the discrete case.